CS327: Compilers

Lab Assignment #4

Intermediate Code Generation

Three-Address Code in Quadruple Format

Submitted by

Bhavya Parmar

Mohit

Surriya Gokul

Tanishq Bhushan Chaudhari

Instructor

Shouvick Mondal

Semester: January–May 2026

Indian Institute of Technology Gandhinagar

Contents

1 Overview **1**

2 Language Design: "Simple" **1**
2.1 Supported Types . . . 1
2.2 Supported Constructs . . . 2
2.3 Relation to Previous Assignment (Cyrix) . . . 2

3 Part 1: Intermediate Code Generation **2**
3.1 Quadruple Representation . . . 2
3.2 Temporary Variable Generation . . . 2
3.3 SDT Actions for Expressions . . . 3
3.4 Type Checking and Implicit Casting . . . 3
3.5 Declaration IR . . . 3
3.6 The Assignment PDF Example . . . 3

4 Part 2: Extended IR Support — Control Flow **4**
4.1 if Statement . . . 4
4.2 if-else Statement . . . 4
4.3 while Loop . . . 4
4.4 for Loop . . . 5
4.5 Implementation Detail: the `for_gbody` Global . . . 5

5 Part 3: Code Output Format **6**
5.1 Output Structure . . . 6
5.2 Quadruple Table Format . . . 6
5.3 Symbol Table Format . . . 6
5.4 Note on Expression Statements . . . 7

6 Part 4: Error Handling **7**
6.1 Design: Error Buffering with IR Suppression . . . 7
6.2 Detected Error Categories . . . 8
6.3 Parser Error Verbosity . . . 8

7 Symbol Table Design **8**
7.1 Structure . . . 8
7.2 Scoping Rules . . . 9

8 Part 5: Optional Machine Code Generation (WSL) **9**
8.1 Target and Toolchain . . . 9
8.2 Lowering Rules (Summary) . . . 9
8.3 Float Handling . . . 9
8.4 Runtime Printing Policy . . . 9
8.5 Scoping Bug and Fix . . . 10
8.6 Error Handling in Codegen . . . 10

9 Test Cases **10**
9.1 Tests 01–10: Standard Programs . . . 10
9.1.1 Test 01 — Basic Arithmetic + Casting (Excerpt) . . . 10
9.1.2 Test 05 — while Loop (Excerpt) . . . 11
9.1.3 Test 07 — Nested if-else (Excerpt) . . . 11
9.1.4 Test 08 — Scoping (Excerpt) . . . 12
9.1.5 Test 09 — break and continue . . . 12
9.2 Tests 11–15: Error Handling . . . 13
9.2.1 Test 11 — Redeclaration + void initialization . . . 13
9.2.2 Test 12 — Use of undeclared variables . . . 13
9.2.3 Test 13 — Control jumps outside loop . . . 14
9.2.4 Test 14 — Unknown characters (lex + parse) . . . 14
9.2.5 Test 15 — Mixed: redeclaration + undeclared variable . . . 14
9.3 Test 16 — Assignment PDF Example . . . 14

10 Implementation Pipeline **15**
10.1 File Structure . . . 15
10.2 Build and Run . . . 15
10.3 Data Flow . . . 15

11 Team Contributions **16**

12 Design Decisions and Justifications **16**

13 Conclusion **16**

1 Overview

This report documents the design and implementation of an **Intermediate Code Generator** for a C-like language called **Simple**, built as the fourth assignment of CS327 Compilers. The compiler translates source programs into **Three-Address Code (TAC)** represented in **Quadruple format**, following the Dragon Book (Aho, Lam, Sethi, Ullman) approach of Syntax-Directed Translation (SDT) with back-patching.

The compiler is implemented using **Flex** (lexer) and **Bison** (parser), and produces:

- A statement-by-statement IR trace with the source line echoed alongside generated quads and a live symbol table snapshot.
- A complete quadruple table at the end of compilation.
- A complete symbol table showing scope, lifetime, initialization status, and last known value for every declared variable.
- Meaningful error diagnostics for semantic errors, with IR suppression on failure.

As an optional extension (for curiosity and verification), the same IR is lowered to x86-64 machine code in WSL and executed to print final variable values. This does not change the language or IR; it is a post-pass.

The pipeline is fully end-to-end: source text → lexer → parser → SDT actions → quad table → formatted output.

2 Language Design: "Simple"

For this assignment the team designed a focused, C-like language called **Simple**. The language was deliberately kept small so that every construct maps cleanly to well-defined TAC patterns, while still being expressive enough to write non-trivial programs.

2.1 Supported Types

`int`, `float`, `void`.
`void` variables can be declared but cannot be assigned or used in expressions or conditions. In the optional machine-code backend, `float` values are represented as **double-precision** for simpler runtime printing.

2.2 Supported Constructs

Category	Constructs
Declarations	`int x;` `int x = expr;`
Arithmetic	`+`, `-`, `*`, `/`, `%` with correct precedence
Unary	Unary minus (`-`), logical NOT (`!`), bitwise NOT (`~`)
Relational	`<`, `>`, `<=`, `>=`, `==`, `!=`
Logical	`&&`, `||`
Assignment	`=`, `+=`, `-=`, `*=`, `/=`
Increment/Decrement	Prefix `++x`, `--x`; Postfix `x++`, `x--`
Selection	`if`, `if-else`, nested `if-else` chains
Iteration	`while`, `for` (with optional init/cond/update)
Jump	`break`, `continue`, `return`, `return expr`
Blocks	Nested { } with full lexical scoping and variable shadowing
Comments	`// ...` and `/* ... */`

2.3 Relation to Previous Assignment (Cyrix)

In Assignment 3, the team implemented a much richer grammar (Cyrix) with lambdas, `foreach`, `match`, `fn`, `var`, power operator (`**`), and more. For A4, the instructor confirmed that only the constructs explicitly listed in the assignment specification need IR generation. Therefore Simple is a clean, focused subset—all constructs in Simple have correct and complete IR generation, whereas the exotic Cyrix constructs (lambdas, generics, etc.) are not required and not present.

3 Part 1: Intermediate Code Generation

3.1 Quadruple Representation

Each IR instruction is stored as a **quadruple**:

$$(\underbrace{\texttt{op}}_{\text{operator}}, \underbrace{\texttt{arg1}}_{\text{first operand}}, \underbrace{\texttt{arg2}}_{\text{second operand}}, \underbrace{\texttt{result}}_{\text{destination}})$$

Unused fields are printed as `-`. The table also carries an `expr` column that renders a human-readable form of each instruction (e.g. `t1=a+b`, `ifFalse t1 goto L2`) for quick reading.

3.2 Temporary Variable Generation

A global counter `tc` generates temporaries `t1`, `t2`, ... in sequential order. Similarly, a counter `lc` generates labels `L1`, `L2`, ...

```
static char *nt(void) { char *b=malloc(16); snprintf(b,16,"t%d",++tc);
    return b; }
static char *nl(void) { char *b=malloc(16); snprintf(b,16,"L%d",++lc);
    return b; }
```

3.3 SDT Actions for Expressions

Every expression non-terminal returns a **place** — either a temporary name or the variable itself — as its semantic value (`$$`). Quads are emitted bottom-up as the parser reduces rules.

Expression	Quads Emitted	Result Place
`A + B`	`(+, A, B, t)`	`t`
`A - B`	`(-, A, B, t)`	`t`
`A * B`	`(*, A, B, t)`	`t`
`A / B`	`(/, A, B, t)`	`t`
`A % B`	`(%, A, B, t)`	`t`
`-E`	`(minus, E, -, t)`	`t`
`!E`	`(!, E, -, t)`	`t`
`x = E`	`(=, E, -, x)`	`x`
`x += E`	`(+, x, E, t)  (=, t, -, x)`	`x`

3.4 Type Checking and Implicit Casting

The compiler performs static type checking and inserts explicit cast quads when needed. Arithmetic and relational operators promote operands to a common numeric type (`float` dominates `int`). Assignments cast the RHS to the LHS type. The cast operations are emitted as quads and shown in the `expr` column as function-like calls.

Cast	Quad Emitted	Expr Column
`int` → `float`	`(itof, x, -, t)`	`t= itof(x)`
`float` → `int`	`(ftoi, x, -, t)`	`t= ftoi(x)`

3.5 Declaration IR

- `int x;` → symbol table entry only; **no quad emitted**. The variable appears in the symbol table with `Init? = no`.
- `int x = E;` → symbol table entry **plus** `(=, E, -, x)` (with implicit cast if needed).
- `void x;` is allowed, but any initialization or assignment to `x` is a semantic error.

This follows the Dragon Book convention: uninitialised declarations simply reserve a name in the symbol table without generating code.

3.6 The Assignment PDF Example

The assignment specification includes the expression `a = b*-c + b*-c` as a reference. Our compiler (test16.c) produces exactly the expected quadruples:

```
| t1= minus c        | minus  | c      | -      | t1     |
| t2=b*t1            | *      | b      | t1     | t2     |
| t3= minus c        | minus  | c      | -      | t3     |
| t4=b*t3            | *      | b      | t3     | t4     |
| t5=t2+t4           | +      | t2     | t4     | t5     |
| a=t5               | =      | t5     | -      | a      |
```

This matches the format shown in the assignment PDF exactly.

4 Part 2: Extended IR Support — Control Flow

Control flow is handled using **back-patching**, following Dragon Book §6.7. Labels and conditional jumps are emitted with placeholder targets (`??`) and later filled in once the target address is known.

4.1 if Statement

`if (C) S`

Quad	Description
`(ifFalse, C, goto, ??)`	Jump if condition false; target back-patched to L_f
[S quads]	Body
`(label,` L_f`, -, -)`	False label

4.2 if-else Statement

`if (C) S else S1`

Quad	Description
`(ifFalse, C, goto, ??)`	Jump to L_f if false
[S quads]	True body
`(goto, ??, -, -)`	Jump to L_{end}
`(label,` L_f`, -, -)`	False label
[S1 quads]	False body
`(label,` L_{end}`, -, -)`	End label

Nested `if-else` chains (e.g. `else if`) are handled naturally because `else if` is just an `if-else` where the else-body is itself another `if-else`. A dedicated LIFO **if-goto stack** (`if_goto_stack`) ensures that the `goto Lend` quad index for each level is stored independently and never overwritten by inner levels.

4.3 while Loop

`while (C) S`

Quad	Description
`(label,` L_s`, -, -)`	Loop start
[C evaluation]	Condition
`(ifFalse, C, goto, ??)`	Exit if false; patched to L_{end}
[S quads]	Body
`(goto,` L_s`, -, -)`	Back-edge
`(label,` L_{end}`, -, -)`	Exit label

4.4 for Loop

```
for (init; cond; update) S
```

The for loop is the most complex construct. Our IR follows the standard Dragon Book layout where the **update code is placed before the body** (using a forward jump), ensuring correct execution order while keeping the condition check at the top of each iteration:

Quad	Description
[init quads]	Initialiser
`(label,` L_{test}`, -, -)`	Condition check point
`(ifFalse, cond, goto, ??)`	Exit if false; patched to L_{end}
`(goto, ??, -, -)`	Forward jump to L_{body}; patched at emit time
`(label,` L_{update}`, -, -)`	Update section
[update quads]	Increment/decrement
`(goto,` L_{test}`, -, -)`	Loop back to condition
`(label,` L_{body}`, -, -)`	Body entry
[S quads]	Body
`(goto,` L_{update}`, -, -)`	Jump to update
`(label,` L_{end}`, -, -)`	Exit label

The `continue` statement jumps to L_{update} (not L_{test}), correctly re-running the update before re-testing the condition. The `break` statement's goto quad index is collected into a per-loop **break buffer** and back-patched to L_{end} when the loop closes.

4.5 Implementation Detail: the `for_gbody` Global

Yacc mid-rule actions occupy exactly one union slot (`$$`). The for-loop's update mid-rule action needs to pass both a label string (`Lupdate`) and an integer (the `goto Lbody` quad index) to the closing action — but a single slot can only hold one. We store the quad index in a **static global** `for_gbody` and the label string in the slot. This is safe because for-loops do not recursively re-enter their own mid-rule actions.

5 Part 3: Code Output Format

5.1 Output Structure

The compiler produces a rich, multi-section output for every source file:

1. **Banner** — compiler name, assignment, and source file name.
2. **Source program** — echoed with line numbers.
3. **Statement-by-statement trace** — for each statement:
   a. Source line (with line number)
   b. Newly generated IR quads in tabular format
   c. Current symbol table state
4. **Final complete quadruple table** — all quads with source line annotation.
5. **Final complete symbol table** — all variables with full lifetime and scope information.
6. **Final variable dump (optional, machine-code run)** — appended at the end of the output file under a banner.

5.2 Quadruple Table Format

Each row has six columns: `No` (sequence number), `expr` (human-readable form), `op`, `arg1`, `arg2`, `result`. The `expr` column is generated by the printer by inspecting the `op` field and formatting accordingly — for example, a binary op becomes `result=arg1 op arg2`; an `ifFalse` quad becomes `ifFalse arg1 goto result`; a cast quad prints as `t= itof(x)` or `t= ftoi(x)`.

```
+======+=====================+===============+===============+===============+===============+=============+
|  No  |        expr         |      op       |     arg1      |     arg2      |    result     |  Src Line   |
+======+=====================+===============+===============+===============+===============+=============+
|    3 | t1=a+b              | +             | a             | b             | t1            | line 4      |
|    4 | c=t1                | =             | t1            | -             | c             | line 4      |
```

5.3 Symbol Table Format

The symbol table is printed after every statement, showing scope depth, declaration line, live-until (either "active" or the closing line number), initialisation status, and the last known value. The value `?` means "unknown" (used when a value is control-flow dependent or not yet assigned).

```
+------------------+---------+-------+-------------+--------------+---------+--------------+
|      Name        |  Type   | Scope | Decl Line   |  Live Until  |  Init?  |    Value     |
+------------------+---------+-------+-------------+--------------+---------+--------------+
|>x                | int     |   0   | line 2      | active       | yes     | 1            |
| x                | int     |   1   | line 5      | line 11      | yes     | 100          |
+------------------+---------+-------+-------------+--------------+---------+--------------+
(> = currently in scope)
```

The `>` marker identifies variables currently in scope. Out-of-scope ("gone") variables from closed blocks are also retained in the table for historical completeness.

5.4 Note on Expression Statements

For bare expression statements (e.g. `z += 4;`) the IR quads are emitted during expression evaluation (before the semicolon reduces), so by the time the `expression_statement` action fires, the quads already exist in the table. The statement-level display therefore shows only the updated symbol table for these statements, rather than repeating the quads. This is a minor display quirk — the quads themselves are always present in the final quadruple table in the correct position.

6 Part 4: Error Handling

6.1 Design: Error Buffering with IR Suppression

Rather than printing errors to `stderr` interleaved with normal output, the compiler uses an **error buffer** design:

```c
typedef struct { char msg[256]; } ErrEntry;
static ErrEntry ERR[MAX_ERRORS];
static int      err_cnt = 0;
static int      suppress_ir = 0;

void add_error(const char *fmt, ...) {
    /* collect error; set suppress_ir = 1 */
}
```

All errors (lex, parse, and semantic) call `add_error()`, which buffers them. The `suppress_ir` flag prevents any IR output from being printed. At the end of compilation, `main()` makes one decision:

- **No errors:** Print full IR output, final quad table, final symbol table, and `[OK]`.
- **Errors:** Print all buffered error messages together in a dedicated **ERRORS DETECTED** section, and `[FAIL]`. IR is suppressed entirely.

This produces clean, unambiguous output: a user either gets full IR or a clean error report — never a mix.

6.2 Detected Error Categories

Error Type	Detection Point	Message Format
Redeclaration in same scope	`sym_decl()`	`[Semantic Error] Line N: 'x' already declared (first at line M)`
Use of undeclared variable	`primary_expression`	`[Semantic Error] Line N: use of undeclared variable 'x'`
Assignment to undeclared variable	`assignment_expression`	`[Semantic Error] Line N: assignment to undeclared variable 'x'`
Void variable initialisation / assignment	`declaration_statement`, `assignment_expression`	`[Semantic Error] Line N: void variable 'x' cannot be initialized`
Void value used in expression or condition	`primary_expression`, condition rules	`[Semantic Error] Line N: void value used in expression`
`break` outside loop	`jump_statement`	`[Semantic Error] Line N: 'break' outside a loop`
`continue` outside loop	`jump_statement`	`[Semantic Error] Line N: 'continue' outside a loop`
Unknown character	`simple.l` rule	`[Lex Error] Line N: unknown character 'X'`
Syntax error	`yyerror()`	`[Parse Error] Line N near 'tok': ...`

6.3 Parser Error Verbosity

The parser uses `%define parse.error verbose` so Bison generates error messages that name the unexpected token and list the tokens that were expected, making syntax errors highly informative.

7 Symbol Table Design

7.1 Structure

```c
typedef struct {
    char name[64], type[16], value[64];
    int  scope;     /* nesting depth at declaration */
    int  init;      /* 0 = declared only, 1 = assigned at least once */
    int  decl;      /* source line of declaration */
    int  end;       /* line where scope closed (-1 = still live) */
    int  active;    /* 1 = currently visible */
} Sym;
```

7.2 Scoping Rules

- Every { block increments the scope depth; every } decrements it and marks all variables at that depth as inactive (setting `end` to the current line).
- **Lookup is innermost-scope-first**: the table is scanned from newest to oldest, so inner declarations shadow outer ones of the same name.
- **Redeclaration** within the *same* scope at the *same* depth is an error. Redeclaration in an *inner* scope (shadowing) is legal and correctly handled.
- **Value tracking**: after a constant assignment (`x = 5`), the value `5` is recorded in the symbol table. After an expression assignment (`x = t3`), the temp name is recorded. This gives the evaluator visibility into what each variable currently holds.
- **Conservative unknowns inside control flow**: assignments inside `if/else` blocks or loops mark the value as `?` because the assignment may or may not execute along all paths. This keeps the symbol table consistent without changing the IR.

8 Part 5: Optional Machine Code Generation (WSL)

This part is not required by the assignment; it was added for curiosity and to validate the IR by executing real code. The backend is a post-pass over the quad table and does not change the grammar or SDT actions.

8.1 Target and Toolchain

The backend emits x86-64 (AMD64) assembly for the System V ABI and is built and run in WSL using GCC. This path is intentionally limited to WSL; native Windows shells are not supported.

8.2 Lowering Rules (Summary)

- Arithmetic: `+,-,*,/` map to `addsd/subsd/mulsd/divsd` for float and `addl/subl/imull/idivl` for int.
- Comparisons: integer compares use `cmpl + setcc`; float compares use `ucomisd + setcc`.
- Control flow: `ifFalse` and `goto` become conditional and unconditional jumps, and `label` emits assembly labels.
- Casts: `itof` uses `cvtsi2sd`, and `ftoi` uses `cvttsd2si`.

8.3 Float Handling

Although the source language uses `float`, the backend stores and computes all float values as **double-precision**. This simplifies `printf` calls in a varargs ABI and avoids mixed-size conversions.

8.4 Runtime Printing Policy

Instead of adding a new print statement to the language, the backend prints a final variable dump after program completion and appends it to the output file. A banner separates the runtime dump from the IR output. Uninitialized variables are printed with a tag, and memory is zero-initialized to keep the dump deterministic.

8.5 Scoping Bug and Fix

The IR stores variables by name only. In the first backend version, a name always mapped to the most recently allocated slot, which broke shadowing in nested scopes (e.g., test 08). The fix resolves a variable by checking its declaration line and scope end against the quad's source line, selecting the innermost symbol active at that line. This preserves correct scoping without modifying the IR.

8.6 Error Handling in Codegen

If any parse or semantic error occurs, the compiler suppresses IR output and does not emit assembly or executables. This prevents running incomplete or inconsistent code.

9 Test Cases

The compiler was tested with **16 programs**: 10 standard programs (tests 01–10), 5 error-handling programs (tests 11–15), and 1 program replicating the assignment PDF example (test 16).

9.1 Tests 01–10: Standard Programs

Test	Description	Key Constructs Exercised
01	Basic arithmetic and assignment	`+`, `-`, `*`, `/`, declarations, `=`
02	Unary operators and compound assignments	`-x`, `+=`, `-=`, `*=`, `/=`
03	Simple `if` (no else)	Conditional jump, back-patching
04	`if-else`	Two-branch selection
05	`while` loop	Loop labels, back-edge, condition
06	`for` loop	All 4 labels, update before body
07	Nested `if-else` (grade classifier)	3-level nesting, if-goto stack
08	Block scoping and variable shadowing	Two `x` variables, scope table
09	`for` loop with `break` and `continue`	Break buffer, continue target
10	Nested loops + `&&` + `+=`	Full integration, 30 quads

9.1.1 Test 01 — Basic Arithmetic + Casting (Excerpt)

Source:

```
int a = 5;
int b = 10;
float c = a + b;
int d = a * b - c;
int e;
e = d / 2;
```

Final Quadruple Table:

```
|    1 | a=5          | =      | 5    | -    | a    | line 2  |
|    2 | b=10         | =      | 10   | -    | b    | line 3  |
|    3 | t1=a+b       | +      | a    | b    | t1   | line 4  |
```

```
|     4 | t2= itof(t1)| itof   | t1    | -     | t2    | line 4   |
|     5 | c=t2        | =      | t2    | -     | c     | line 4   |
|     6 | t3=a*b      | *      | a     | b     | t3    | line 5   |
|     7 | t4= itof(t3)| itof   | t3    | -     | t4    | line 5   |
|     8 | t5=t4-c     | -      | t4    | c     | t5    | line 5   |
|     9 | t6= ftoi(t5)| ftoi   | t5    | -     | t6    | line 5   |
|    10 | d=t6        | =      | t6    | -     | d     | line 5   |
|    11 | t7=d/2      | /      | d     | 2     | t7    | line 7   |
|    12 | e=t7        | =      | t7    | -     | e     | line 7   |
```

Final variable dump (runtime):

```
a = 5
b = 10
c = 15.000000
d = 35
e = 17
```

9.1.2 Test 05 — while Loop (Excerpt)

Source:

```
int i = 0;
int sum = 0;
while (i < 5) {
    sum = sum + i;
    i = i + 1;
}
```

Final Quadruple Table:

```
|     3 | label L1           | label    | L1    | -     | -     | line 4   |
|     4 | t1=i<5             | <        | i     | 5     | t1    | line 4   |
|     5 | ifFalse t1 goto L2| ifFalse | t1    | goto  | L2    | line 4   |
|     6 | t2=sum+i           | +        | sum   | i     | t2    | line 5   |
|     7 | sum=t2             | =        | t2    | -     | sum   | line 5   |
|     8 | t3=i+1             | +        | i     | 1     | t3    | line 6   |
|     9 | i=t3               | =        | t3    | -     | i     | line 6   |
|    10 | goto L1            | goto     | L1    | -     | -     | line 7   |
|    11 | label L2           | label    | L2    | -     | -     | line 7   |
```

Final variable dump (runtime):

```
i = 5
sum = 10
```

9.1.3 Test 07 — Nested if-else (Excerpt)

Source:

```
int marks = 82;
int grade;
if (marks >= 90)        grade = 10;
else if (marks >= 80)   grade = 9;
else if (marks >= 70)   grade = 8;
else                    grade = 7;
```

Final Quadruple Table (control-flow quads only):

```
|     2 | t1=marks>=90        | >=      | marks| 90   | t1   |
|     3 | ifFalse t1 goto L1  | ifFalse | t1   | goto | L1   |
|     4 | grade=10            | =       | 10   | -    | grade|
|     5 | goto L6             | goto    | L6   | -    | -    |  <- patched
|     6 | label L1            | label   | L1   | -    | -    |
|     7 | t2=marks>=80        | >=      | marks| 80   | t2   |
|     8 | ifFalse t2 goto L2  | ifFalse | t2   | goto | L2   |
|     9 | grade=9             | =       | 9    | -    | grade|
|    10 | goto L5             | goto    | L5   | -    | -    |  <- patched
|    11 | label L2            | label   | L2   | -    | -    |
     ... (continues to L3, L4, L5, L6)
|    18 | label L4            | label   | L4   | -    | -    |
|    19 | label L5            | label   | L5   | -    | -    |
|    20 | label L6            | label   | L6   | -    | -    |
```

Each `goto` is correctly back-patched to its matching end-label via the LIFO `if_goto_stack`.

Final variable dump (runtime):

```
marks = 82
grade = 9
```

9.1.4 Test 08 — Scoping (Excerpt)

Source:

```
int x = 1;
int y = 2;
{
    int x = 100;    /* shadows outer x */
    y = x + y;     /* uses inner x=100 */
    { int z = x * 2; y = y + z; }
}
int result = x + y;  /* uses outer x=1 */
```

The symbol table snapshot after the outer block closes correctly shows the inner `x` (scope 1) as `gone` and the outer `x` (scope 0) as `active`. The quad `(+, x, y, t4)` for `result` uses the outer `x=1`.

Final variable dump (runtime):

```
x = 1
y = 302
result = 303
```

9.1.5 Test 09 — break and continue

Source:

```
for (i = 0; i < 10; i = i + 1) {
    if (i == 5) break;
    if (i == 2) continue;
    sum = sum + i;
}
```

Key quads:

```
|   14 | goto L6   |  <- break:    jumps to Lend (L6), back-patched
|   18 | goto L2   |  <- continue: jumps to Lupdate (L2)
```

Final variable dump (runtime):

```
sum = 8
i = 5
```

9.2 Tests 11–15: Error Handling

The error-handling tests explicitly validate that the compiler detects semantic/lex/parse errors, buffers them, and suppresses IR output. The table below summarizes the intent, followed by the exact sources and the error messages observed in the output files.

Test	Error Scenario	Errors Detected	Outcome
11	Redeclaration in same scope + void init	2 semantic errors	`[FAIL]`, IR suppressed
12	Use of undeclared variables	2 semantic errors	`[FAIL]`, IR suppressed
13	`break`/`continue` outside loop	2 semantic errors	`[FAIL]`, IR suppressed
14	Unknown character in expression	1 lex + 1 parse error	`[FAIL]`, IR suppressed
15	Mixed: inner redeclaration + undeclared var	2 semantic errors	`[FAIL]`, IR suppressed

9.2.1 Test 11 — Redeclaration + void initialization

Source:

```
int x = 5;
int y = 10;
int x = 20;
void z = x + y;
```

Errors reported:

```
[Semantic Error] Line 4: 'x' already declared in this scope (first declared at
    line 2)
[Semantic Error] Line 5: void variable 'z' cannot be initialized
```

9.2.2 Test 12 — Use of undeclared variables

Source:

```
int a = 10;
int b = a + c;
int d;
d = b * e;
```

Errors reported:

```
[Semantic Error] Line 3: use of undeclared variable 'c'
[Semantic Error] Line 5: use of undeclared variable 'e'
```

9.2.3 Test 13 — Control jumps outside loop

Source:

```
int x = 5;
break;
int y = 10;
continue;
```

Errors reported:

```
[Semantic Error] Line 3: 'break' outside a loop
[Semantic Error] Line 5: 'continue' outside a loop
```

9.2.4 Test 14 — Unknown characters (lex + parse)

Source:

```
int a = 5;
int b = a @ 3;
int c = b # 2;
```

Errors reported:

```
[Lex Error]    Line 3: unknown character '@'
[Parse Error]  Line 3 near '3': syntax error, unexpected I_CONSTANT, expecting ';'
```

9.2.5 Test 15 — Mixed: redeclaration + undeclared variable

Source:

```
int a = 1;
int b = 2;
{
    int a = 10;
    int a = 20;
    b = a + q;
}
int result = a + b;
```

Errors reported:

```
[Semantic Error] Line 6: 'a' already declared in this scope (first declared at
    line 5)
[Semantic Error] Line 7: use of undeclared variable 'q'
```

Because errors suppress IR, the machine-code backend is also skipped and no assembly or executable is produced for these tests.

9.3 Test 16 — Assignment PDF Example

Source:

```
int a = 1;
int b = 2;
int c = 3;
a = b * -c + b * -c;
```

Final Quadruple Table:

```
|    4 | t1= minus c   | minus | c    | -    | t1   | line 4  |
|    5 | t2=b*t1       | *     | b    | t1   | t2   | line 4  |
|    6 | t3= minus c   | minus | c    | -    | t3   | line 4  |
|    7 | t4=b*t3       | *     | b    | t3   | t4   | line 4  |
|    8 | t5=t2+t4      | +     | t2   | t4   | t5   | line 4  |
|    9 | a=t5          | =     | t5   | -    | a    | line 4  |
```

Final variable dump (runtime):

```
a = -12
b = 2
c = 3
```

This is an exact match of the quadruple table shown in the assignment specification.

10 Implementation Pipeline

10.1 File Structure

File	Role	Lines
`simple.l`	Flex lexer	≈100
`simple.y`	Bison parser + SDT + all IR logic	≈1000
`Makefile`	Build automation	40
`tests/test01--10.c`	Standard test programs	10 files
`tests/test11--15.c`	Error-handling test programs	5 files
`tests/test16.c`	Assignment PDF example	1 file
`outputs/output01--16.txt`	Outputs	16 files

10.2 Build and Run

```
sudo apt install bison flex   # one-time install
make                          # build the compiler
make tests                    # run all 16 tests, save outputs
make tests-asm                # generate .s + .exe for all valid tests (WSL)
make tests-run                # run .exe and append runtime output (WSL)
make clean                    # remove build artifacts
```

10.3 Data Flow

$$\texttt{source.c} \xrightarrow{\text{flex (simple.l)}} \text{token stream} \xrightarrow{\text{bison (simple.y)}} \text{LALR(1) parse + SDT} \xrightarrow{\text{emit_q()}} \text{quad table}$$
$$\xrightarrow{\text{print_summary()}} \text{formatted IR output} \xrightarrow{\text{codegen (WSL)}} \texttt{.s} \rightarrow \texttt{gcc} \rightarrow \texttt{.exe} \rightarrow \text{final variable dump}$$

11 Team Contributions

Team Member	Contribution
Bhavya Parmar	*Rewrote the previous Cyrix grammar for this assignment (removed extra constructs).*
Mohit	*SDT for basic constructs; type checking and implicit casting logic; IR output formatting; machine-code generation backend.*
Surriya Gokul	*Control-flow SDT (if/else/loops) and back-patching logic checks.*
Tanishq Bhushan Chaudhari	*Test suite creation (tests 01–16), output verification, and report write-up polishing.*

12 Design Decisions and Justifications

1. **Named non-terminal `if_prefix` for if/if-else.** Writing two separate rules both starting with `IF '(' expression ')'` caused 22 reduce/reduce conflicts in Bison (the parser cannot tell which rule to start reducing until it sees `ELSE` or not). Factoring the shared prefix into `if_prefix` reduces this to the single expected dangling-else shift/reduce conflict, which Bison resolves correctly by default (shift = attach else to nearest if).

2. **LIFO `if_goto_stack` for nested if-else.** The `goto Lend` quad index from the outer if-else must survive while the inner if-else runs. A single global is overwritten; a stack preserves each level independently.

3. **`for_gbody` global for the for-loop.** A single mid-rule action slot cannot carry both a `char*` (Lupdate label) and an `int` (goto Lbody index). The static global `for_gbody` carries the integer, while the slot carries the string. This is safe because for-loop mid-rule actions do not re-enter each other recursively.

4. **Error buffering and IR suppression.** Rather than printing errors to stderr mid-output, all errors are buffered. If any error occurred, the final output is a clean error report with no partial IR. This prevents the evaluator from seeing corrupted or misleading intermediate code.

5. **Dragon Book quadruple convention, no optimisation.** All IR is emitted exactly as the Dragon Book prescribes, with no constant folding, dead-code elimination, or any other optimisation. This ensures the output is predictable and matches lecture slide conventions.

6. **`%option yylineno` in Flex.** Rather than manually maintaining a line counter in the lexer, `%option yylineno` asks Flex to track line numbers automatically. This eliminates the `yylineno` redefinition error that occurs when the lexer and the Flex-generated `lex.yy.c` both try to define the same variable.

13 Conclusion

The Simple compiler correctly implements the four required parts of Assignment 4, plus an optional Part 5:

- **Part 1** — IR generation for arithmetic, assignment, unary operators, and temporary variable generation, verified against the assignment PDF example.
- **Part 2** — Extended IR for `if`, `if-else`, `while`, `for`, `break`, and `continue`, using Dragon Book back-patching.
- **Part 3** — Clean tabular output with an `expr` column, a live symbol table after each statement, and a final complete quad table.
- **Part 4** — Nine distinct error categories detected and reported clearly, with IR suppressed on any error.
- **Part 5 (optional)** — A WSL-only machine-code backend that executes the quads and prints final variable values for verification.

The compiler has been tested with 16 programs (10 standard + 5 erroneous + 1 PDF example) and all outputs have been verified correct. The optional machine-code path further confirms correctness by matching runtime final values with the IR semantics.